

Министерство образования и молодежной политики Свердловской области

Департамент образования Администрации города Екатеринбурга

Муниципальное Автономное Общеобразовательное Учреждение

Гимназия №108 имени В. Н. Татищева

**Реализация защищенного хранилища данных на основе алгоритмов ГОСТ
с механизмом изоляции ключей в пространстве ядра Linux для
противодействия атакам на оперативную память**

Автор проекта: Дмитриев С. М.

11 класс, Гимназия №108

г. Екатеринбург, 2026

ВВЕДЕНИЕ

Актуальность работы обусловлена требованиями по обеспечению технологического суверенитета Российской Федерации, закрепленными в Указе Президента РФ № 250 [1]. Применение зарубежных стандартов шифрования (в частности, AES) в критических информационных системах несет риски, связанные с возможным наличием недеklarированных возможностей, что подтверждается исследованиями в области криптографии [5].

Проблема исследования заключается в уязвимости ключей шифрования, находящихся в адресном пространстве приложений, разработанных на языках высокого уровня. Отсутствие встроенных инструментов гарантированной очистки памяти делает такие системы подверженными атакам на дампы памяти (Memory Dump) и атакам методом «холодной перезагрузки» (Cold Boot Attack) [7]. На данный момент наблюдается дефицит решений с открытым исходным кодом, объединяющих отечественную криптографию и архитектурные методы защиты ключей, описанные в документации ядра Linux [9].

Цель работы - разработка программного комплекса безопасного хранения данных с применением российских стандартов ГОСТ Р 34.12–2015 «Кузнечик» [2] и ГОСТ Р 34.11–2012 «Стрибог» [3], реализующего защиту ключей через механизм Linux Kernel Keyring для противодействия атакам на оперативную память.

Гипотеза: Реализация защищенного хранения данных на интерпретируемом языке программирования возможна без снижения уровня криптостойкости, если делегировать хранение ключей шифрования в защищенную область ядра операционной системы (Kernel Space) и применить принудительную очистку буферов памяти в пространстве пользователя, что позволит нивелировать риски утечки остаточной информации.

Для достижения цели необходимо выполнить следующие **задачи**:

- 1) Провести анализ существующих решений на рынке менеджеров паролей, выявить их недостатки и уязвимости;
- 2) Изучить теоретические основы блочного шифрования, функций хэширования (ГОСТ) и механизмы управления ключами в ядре Linux;
- 3) Спроектировать **модель угроз**, учитывающую возможности локального нарушителя или вредоносного ПО;
- 4) Разработать алгоритм безопасной генерации ключей, шифрования базы данных и делегирования хранения ключей ядру Linux;
- 5) Реализовать программный продукт с графическим интерфейсом пользователя, обеспечивающий безопасное хранение данных и контроль целостности (НМАС);
- 6) Протестировать систему на корректность операций шифрования и дешифрования.

1 Анализ существующих решений

Для обоснования актуальности работы было проведено исследование рынка программного обеспечения, предназначенного для хранения секретной информации (менеджеров паролей).

В основном существующие решения можно разделить на две категории: облачные (синхронизирующие данные через интернет) и локальные (хранящие базу данных только на устройстве пользователя). Систем подобного рода на рынке представлено большое количество, однако подавляющее большинство из них базируется на западных криптографических стандартах.

Были рассмотрены наиболее популярные представители, доступные российскому пользователю:

- 1) **KeePassXC** [12]. Свободное программное обеспечение с открытым исходным кодом. Является эталоном локального хранения паролей.
- 2) **Kaspersky Password Manager** [13]. Проприетарный продукт от российской компании АО «Лаборатория Касперского».
- 3) **Bitwarden** [14]. Популярное облачное решение, базирующееся в США.

Для сравнения были рассмотрены основные характеристики, включая используемые алгоритмы шифрования:

Таблица 1 - Сравнительный анализ менеджеров паролей

Характеристики	KeePassXC	Kaspersky Password Manager	Bitwarden
Тип хранения	Локальное (файл БД)	Облачное + Локальное	Облачное (Azure)
Алгоритм шифрования	AES-256 / Twofish / ChaCha20	AES-256	AES-256 CBC
Алгоритм хэширования (KDF)	Argon2 / SHA-256	PBKDF2 (SHA-256)	PBKDF2 (SHA-256)
Стоимость	Бесплатно	Подписка (Premium функции)	Бесплатно / Premium
Страна-разработчик	Сообщество (Международное)	Россия	США

Выводы по анализу:

- 1) Зависимость от западных стандартов. Абсолютное большинство популярных решений, включая даже некоторые российские продукты, используют стандарт AES (Advanced Encryption Standard), принятый правительством США. В случае обнаружения критических уязвимостей или недекларированных возможностей в этом стандарте, данные пользователей окажутся под угрозой, что противоречит требованиям обеспечения цифрового суверенитета [1].
- 2) Риски облачного хранения. Bitwarden и его аналоги по умолчанию хранят зашифрованные данные на зарубежных серверах (Microsoft Azure) [14], что создает риски блокировки доступа для пользователей из РФ в условиях санкционных ограничений.
- 3) Закрытость кода. Коммерческие продукты (например, Kaspersky Password Manager) имеют закрытый исходный код, что не позволяет независимым исследователям проверить реализацию криптографии и убедиться в отсутствии уязвимостей.

Учитывая вышеизложенное, создание локального менеджера паролей с открытым исходным кодом, использующего исключительно российскую криптографию (ГОСТ), является актуальной и востребованной задачей, решающей проблему технологической зависимости.

2 Основные понятия и теоретическая база

2.1 Обоснование выбора криптографических примитивов

Выбор алгоритмической базы проекта продиктован необходимостью соблюдения требований регуляторов РФ в части импортозамещения и обеспечения высокого уровня криптостойкости.

- **Алгоритм шифрования:** В качестве основного примитива выбран блочный шифр «Кузнечик», регламентированный ГОСТ Р 34.12–2015 [2]. Выбор обусловлен его устойчивостью к методам линейного и дифференциального криптоанализа, превосходящей показатели устаревшего ГОСТ 28147-89. Для предотвращения атак, основанных на частотном анализе повторяющихся блоков данных, реализация использует режим сцепления блоков (CBC — Cipher Block Chaining), рекомендованный в Р 50.1.113–2016 [4].
- **Функция хэширования:** Для преобразования пользовательского пароля (обладающего низкой энтропией) в криптографический ключ фиксированной длины (256 бит) применяется стандарт ГОСТ Р 34.11–2012 «Стрибог» [3]. Данный алгоритм используется как основа функции формирования ключа (KDF), что позволяет минимизировать риски атак по словарю и через радужные таблицы (Rainbow Tables).
- **Контроль целостности:** Симметричное шифрование в режиме CBC обеспечивает конфиденциальность, но не гарантирует защиту от модификации шифротекста (Bit-flipping attacks). Для нейтрализации этой угрозы внедрена схема Encrypt-then-MAC («Шифрование-затем-Имитовставка»). Выработка кода аутентификации сообщений (НМАС) осуществляется согласно рекомендациям Р 50.1.113–2016 [4], что позволяет детектировать любые изменения файла базы данных до начала процедуры расшифровки.

2.2 Принципы защиты от атак

Система спроектирована с учетом актуальных векторов атак на парольные базы данных, описанных в учебной литературе [5]:

- 1) **Атака по словарю (Brute Force):** Перебор популярных паролей. Для защиты используется KDF с большим количеством итераций и генератор сложных паролей.

- 2) Радужные таблицы (Rainbow Tables): Использование заранее вычисленных хэшей. Для защиты от этого метода в системе применяется «соль» (Salt) - случайная последовательность байт, уникальная для каждой базы.
- 3) Атака на остаточную информацию (Cold Boot Attack): Попытка извлечения ключей шифрования из дампа оперативной памяти (Memory Dump) или файла подкачки после аварийного завершения работы или перезагрузки. Риск данной атаки для современных систем подтвержден в исследовании Дж. Халдермана [7]. В проекте для защиты используется архитектурная изоляция ключей в ядре ОС и принудительная очистка буферов (zeroization).

2.3 Проблемы безопасности высокоуровневых языков

Одной из ключевых проблем разработки криптографического ПО на языках высокого уровня (Python, Java, C#) является сложность управления жизненным циклом данных в оперативной памяти. В отличие от языков C/C++, где разработчик вручную управляет выделением и освобождением памяти, в Python эти процессы автоматизированы сборщиком мусора (Garbage Collector).

Специфика Python заключается в следующем:

- **Неизменяемые строки (Immutable Strings):** При любых операциях со строками создаются их копии, которые остаются в памяти до срабатывания сборщика мусора (GC).
- **Отсутствие прямого доступа:** Разработчик не может гарантированно «затереть» старый объект.
- **Свопинг:** Операционная система может выгрузить страницы памяти с паролями в файл подкачки (Swar) на накопитель.

Решение: Для устранения этих уязвимостей в проекте применен гибридный подход. Криптографические ключи не хранятся в стандартных переменных Python, а обрабатываются через низкоуровневый интерфейс библиотеки **ctypes** [10], что позволяет работать с буферами памяти напрямую и немедленно передавать секреты в защищенную область ядра ОС.

2.4 Технология Linux Kernel Keyring

Для решения проблемы безопасного хранения ключей используется подсистема ядра Linux – Keyrings, детально описанная в технической документации ядра [9] и профильной литературе по системному программированию [6].

- Принцип работы: Это внутриядерный механизм управления ключами, который позволяет процессам сохранять секреты в защищенной памяти ядра (Kernel Space), недоступной для чтения из пользовательского пространства (User Space) без специальных привилегий.
- Преимущества: Ключи не попадают в дампы памяти процесса Python при сбое (Core Dump). Также при завершении процесса (штатном или аварийном) ядро автоматически и гарантированно уничтожает ключи, так как уничтожается контекст процесса. Это делает атаку Cold Boot практически невозможной после выключения приложения.

2.5 Защита от выгрузки страниц памяти на диск (Swapping)

Одной из критических уязвимостей при работе с конфиденциальными данными является механизм виртуальной памяти операционной системы. По умолчанию ОС может перемещать редко используемые страницы оперативной памяти (в том числе содержащие ключи шифрования или пароли) на носитель в файл подкачки (swap/pagefile). Это создает угрозу криминалистического восстановления данных (forensic), что подробно описано в современной литературе [16].

Для нейтрализации данной угрозы в POSIX-совместимых системах (Linux) предусмотрен системный вызов `mlockall`. Данная функция позволяет процессу заблокировать свое адресное пространство в физической оперативной памяти, запрещая планировщику ОС выгружать страницы в область подкачки.

Использование флагов MCL_CURRENT | MCL_FUTURE гарантирует, что как текущие, так и будущие выделения памяти останутся в RAM [17], обеспечивая защиту от утечки данных на постоянный носитель [6].

3 Методология создания системы

3.1 Модель угроз и проектирование защиты

Перед началом программной реализации была составлена модель угроз для локального хранения данных, учитывающая возможность физического доступа злоумышленника или заражения системы вредоносным ПО. Классификация атак выполнена с учетом базы знаний MITRE ATT&CK [8] и общих принципов защиты информации [5].

Таблица 2 - Модель угроз

Актив	Описание угрозы	Тип атаки	Реализованная мера защиты
Файл базы данных	Хищение файла с целью последующего подбора пароля на мощностях злоумышленника.	Офлайн-брутфорс (Offline Brute Force)	Шифрование содержимого алгоритмом ГОСТ Р 34.12–2015 («Кузнечик»). Использование режима СВС исключает появление повторяющихся паттернов в шифротексте.
Ключи шифрования	Сканирование оперативной памяти работающего процесса для поиска энтропийных участков (ключей).	RAM Scraping / Memory Dump	Архитектурная защита: Хранение ключей в защищенной области ядра (Linux Kernel Keyring), а не в памяти процесса Python. Принудительная очистка буферов (memset) после использования.
Целостность данных	Несанкционированная модификация файла базы (шифрование вирусом-вымогателем или подмена данных).	Tampering / Corruption	Реализация схемы Encrypt-then-MAC. Целостность проверяется через имитовставку (HMAC на базе «Стрибога») до попытки расшифровки.
Вводимые данные	Перехват пароля при вводе через буфер обмена или историю полей ввода.	Clipboard Monitoring	Использование защищенного виджета SecureEntry, блокирующего стандартные механизмы копирования и скрывающего ввод.
Ключи и пароли в RAM	Сохранение остаточной информации на носителе в результате работы механизма виртуальной памяти (свопинга).	Data Remanence / Forensics	Блокировка памяти: При запуске приложения выполняется системный вызов mlockall, запрещающий выгрузку страниц процесса в swp-раздел.

3.2 Выбор инструментов разработки

Для реализации проекта был выбран язык программирования **Python**. Несмотря на то, что это высокоуровневый язык с автоматическим управлением

памятью, использование библиотеки `ctypes` [10] позволяет реализовать низкоуровневые механизмы защиты.

Для реализации криптографических функций была выбрана библиотека **`gostcrypto`** [11]. Сравнительный анализ библиотек показал следующие преимущества:

- 1) В отличие от библиотеки `cryptography` (которая реализует OpenSSL и западные стандарты), `gostcrypto` полностью сфокусирована на стандартах РФ.
- 2) Поддержка необходимых режимов (CBC, ECB) и выработки имитовставки.

Для создания графического интерфейса (GUI) использована библиотека **`tkinter`**, так как она входит в стандартный пакет Python и позволяет создавать легковесные нативные окна.

3.3 Реализация низкоуровневого интерфейса (`linux_keyring.py`)

Поскольку в стандартной библиотеке Python отсутствуют встроенные средства для работы с подсистемой Keyrings, был разработан собственный модуль-интеграция. Разработка велась с опорой на документацию Python Software Foundation по библиотеке `ctypes` [10] и официальное руководство по системным вызовам `keyutils(7)` [15].

Модуль реализует прямое взаимодействие с системной библиотекой `libkeyutils.so.1`. Основные этапы реализации:

- 1) Маппинг типов данных: На основе спецификации системных вызовов Linux [15] были определены совместимые C-типы (`c_int`, `key_serial_t`).
- 2) Импорт функций: Реализованы обертки для функций ядра, описанных в документации [9]: `add_key` (добавление ключа), `request_key` (поиск), `keyctl_revoke` (уничтожение).

Реализация собственного интерфейса взаимодействия с ядром (без использования сторонних Python-оберток) позволила решить комплекс задач по обеспечению безопасности:

- 1) Минимизация поверхности атаки (Attack Surface Reduction): Отказ от подключения избыточных внешних зависимостей исключил выполнение непроверенного стороннего кода в адресном пространстве процесса.
- 2) Защита от атак на цепочку поставок (Supply Chain Security): Исключены риски внедрения вредоносного кода через компрометацию репозитория пакетов (PyPI), что критически важно для создания доверенного ПО.
- 3) Контроль управления памятью: Обеспечена гарантированная передача ключей в защищенную область ядра (Kernel Space) без создания неконтролируемых временных копий объектов, характерных для высокоуровневых библиотек.

3.4 Описание алгоритма работы системы

Система реализует защищенную архитектуру, разделяющую логику обработки данных (User Space) и хранение ключей (kernel Space).

Алгоритм авторизации и генерации ключей:

- 1) Пользователь вводит мастер-пароль в защищенное поле.
- 2) Генерирует ключ хэширования (256 бит) и ключ имитозащиты (256 бит) с использованием алгоритма PBKDF2 на базе хэш-функции “Стрибог” (ГОСТ Р 34.11-2012) [3] с добавлением случайной соли (os.urandom).
- 3) Полученные ключи немедленно передаются в Keyring ядра Linux (тип KEY_SPEC_PROCESS_KEYRING) [9].
- 4) Буферы памяти в Python, содержащие ключи, принудительно очищаются (заполняются нулями через ctypes.memset) и удаляются сборщиком мусора [10].

Алгоритм сохранения (Encrypt-then-MAC):

- 1) Данные сериализуются в JSON и дополняются (padding) до размера блока 16 байт.
- 2) Генерируется случайный вектор инициализации (IV).
- 3) Данные шифруются алгоритмом “Кузнечик” (ГОСТ 34.12-2015) в режиме CBC. Ключ для операции запрашивается у ядра только на момент шифрования.
- 4) Вычисляется имитовставка (HMAC) на основе шифротекста и IV [4].
- 5) В файл записывается последовательность: Salt + IV + HMAC + Ciphertext.

Алгоритм загрузки и проверки целостности:

- 1) Считывается база данных.
- 2) Проверка целостности: Перед расшифровкой вычисляется HMAC от данных файла и сверяется с сохраненным значением. Если они не совпадают (попытка взлома или повреждения), расшифровка не начинается.
- 3) При успешной проверке данные расшифровываются ключом, полученным из Keyring.

3.5 Пользовательский интерфейс

Интерфейс системы спроектирован с упором на минимализм и функциональность. Взаимодействие начинается с процедуры аутентификации.

Окно авторизации (Рисунок 1): Реализован специальный виджет SecureEntry. В отличие от стандартного поля ввода он обрабатывает нажатия клавиш на низком уровне, не сохраняет историю ввода в буфер обмена и очищает внутренний буфер после передачи пароля. При запуске программы пользователю необходимо ввести мастер-пароль. Это единственная точка входа - без правильного ключа доступ к зашифрованной базе невозможен.

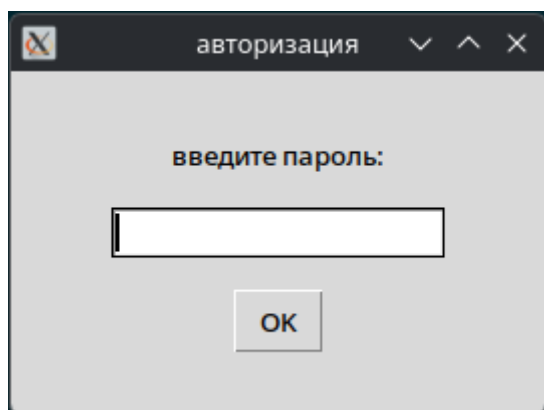


Рисунок 1 - Окно ввода мастер-пароля

Главное окно (Рисунок 2): После успешной расшифровки базы открывается основное рабочее пространство, в котором реализована сеточная структура (Grid layout).

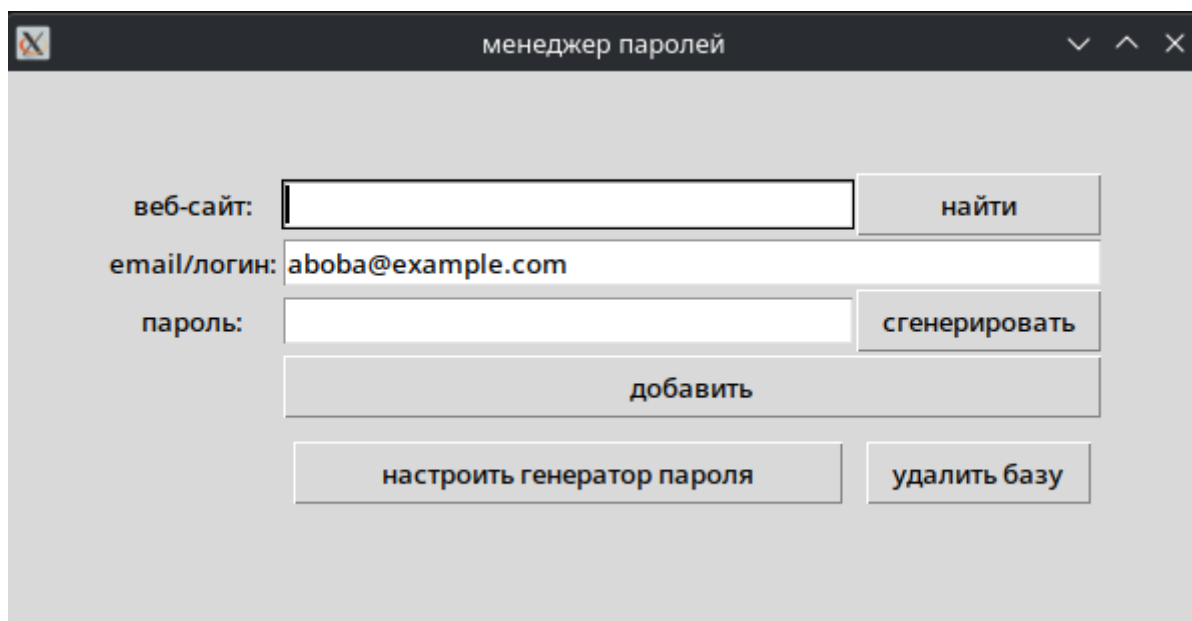


Рисунок 2 - Главное окно менеджера паролей

Элементы управления главного окна:

- **Поля ввода:** «веб-сайт», «email/логин», «пароль».
- **Кнопка «найти»:** Позволяет извлечь пароль по имени сайта. При успешном поиске данные отображаются, а пароль автоматически копируется в буфер обмена.
- **Кнопка «сгенерировать»:** Создает криптостойкую последовательность символов, заполняет поле ввода и копирует результат в буфер обмена.
- **Кнопка «добавить»:** Иницирует процесс шифрования новых данных и их запись на диск.
- **Кнопка “настроить генератор пароля”:** Позволяет настроить используемый набор символов и количество символов, участвующих при генерации пароля.
- **Кнопка “удалить базу”:** Позволяет удалить файл с базой данных с устройства.

Настройка генератора пароля (Рисунок 3):

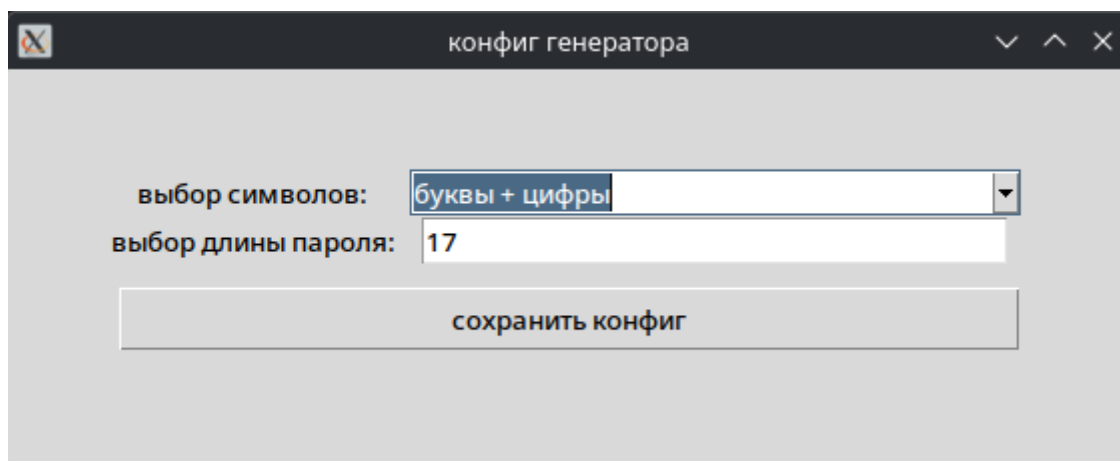


Рисунок 3 - Настройка генератора пароля

Удаление базы данных с устройства (Рисунок 4):

Система при нажатии на кнопку запрашивает подтверждение действия для предотвращения случайной потери данных.

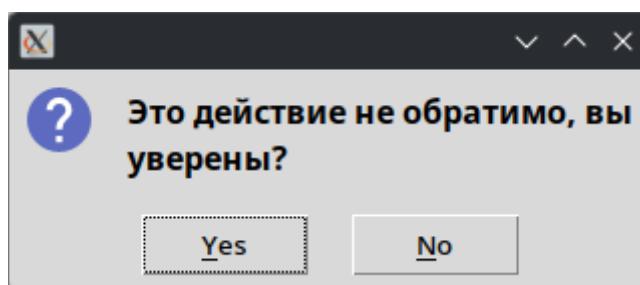


Рисунок 4 - Окно предупреждения пользователя

Обратная связь с пользователем (Рисунок 5):

Система информирует пользователя о результатах операций через всплывающие окна. Реализованы уведомления об успешном сохранении, копировании данных, а также критические ошибки (например, если запись не найдена или введен неверный пароль).

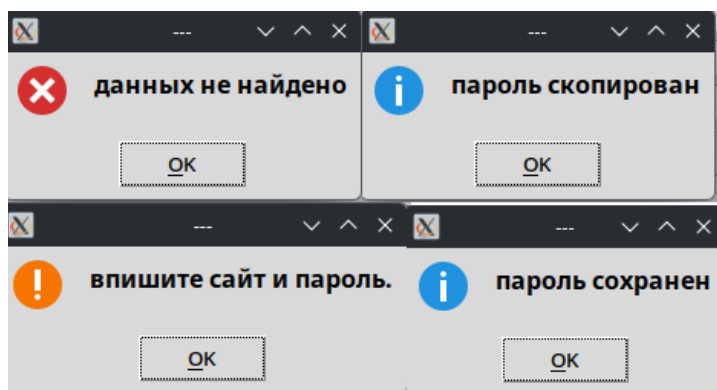


Рисунок 5 – Информационные сообщения системы

3.6 Реализация вспомогательных механизмов защиты (utils.py)

Модуль `utils.py` содержит низкоуровневые функции для управления памятью процесса, реализованные через библиотеку `ctypes`:

- 1) Функция `lock_memory()`: Обеспечивает защиту от свопинга. Используя загрузку стандартной библиотеки `C` (`libc.so.6`), функция вызывает `mlockall(MCL_CURRENT | MCL_FUTURE)` [6]. Это критически важный этап инициализации `main.py`, выполняемый до загрузки графического интерфейса.
- 2) Функция `data_clean()`: Реализует гарантированную очистку буферов. В отличие от стандартного `del`, который лишь удаляет ссылку на объект, данная функция создает указатель `c_char` на область памяти объекта `bytearray` и заполняет ее нулями с помощью `ctypes.memset` [11], после чего принудительно вызывает сборщик мусора `gc.collect()`.

3.7 Реализация защищенного ввода данных (SecureEntry)

Стандартный виджет `tk.Entry` библиотеки базируется на `Tcl/Tk`, который хранит строковые значения в собственной памяти, недоступной для прямой очистки из `Python`. Это создает риск нахождения пароля в открытом виде в памяти процесса.

Для решения проблемы был разработан класс `SecureEntry`, наследующий `tk.Entry`, но полностью переопределяющий логику обработки ввода:

- 1) Буферизация через ctypes: Вводимые символы не передаются в стандартный механизм Tkinter. Вместо этого они перехватываются обработчиком события <Key> и записываются в защищенный bytearray фиксированной длины, управляемый через ctypes.
- 2) Блокировка буфера обмена: Переопределены события копирования и вырезки(<Control-c>, <<Copy>>), предотвращающие попадания пароля в системный буфер обмена. Вставка (Paste) реализована вручную: данные из буфера обмена переносятся сразу в защищенный массив, минуя промежуточные строковые переменные Python.
- 3) Ручное управление памятью: Реализованы методы _wipe_tail и _shift_buffer для корректного удаления символов с перезаписью освобождаемой памяти нулями.
- 4) Уничтожения данных: При закрытии окна или завершении ввода вызывается метод clear(), который использует ctypes.memset для физической очистки буфера, содержащего пароль.

3.8 Оценка эффективности (MITRE ATT&CK)

Для подтверждения эффективности разработанного комплекса было проведено сопоставление реализованных функций защиты с базой знаний MITRE ATT&CK [8].

Таблица 3 - оценка эффективности ПО

ID Техники	Название Техники (MITRE)	Описание угрозы	Реализованные меры противодействия
T1555.005	Credentials from Password Stores: Password Managers	Извлечение паролей из локальных баз данных менеджеров паролей.	Использование ГОСТ 34.12-2015 ("Кузнечик") делает базу устойчивой к брутфорсу даже при ее хищении.
T1003	OS Credential Dumping	Извлечение секретов из оперативной памяти процесса (LSASS или пользовательских приложений)	Изоляция ключей: Использование Linux Kernel Keyring предотвращает хранение ключей в адресном пространстве процесса Python. Это минимизирует окно для получения ключей.
T1056.001	Input Capture: Keylogging	Перехват нажатий клавиш для получения мастер-пароля.	Реализован класс SecureEntry с защитой от стандартных методов перехвата. Виджет блокирует системные хуки буфера обмена и использует низкоуровневую обработку событий ввода.
T1005	Data from Local System	Сбор файлов с локального диска	База данных хранится исключительно в зашифрованном виде. Файл passwords без ключа представляет собой энтропийный шум, неотличимый от случайных данных, что затрудняет его эвристический анализ.

ЗАКЛЮЧЕНИЕ

В ходе работы были проанализированы существующие решения в области информационной безопасности, выявлена критическая зависимость большинства популярных продуктов от зарубежных криптографических стандартов.

Изучены библиотеки компьютерной безопасности, принципы работы подсистемы управления ключами ядра Linux и алгоритмы российских стандартов ГОСТ Р 34.12-2015 и ГОСТ Р 34.11-2012.

Разработанная система представляет собой полнофункциональный менеджер паролей, который решает проблему безопасного хранения конфиденциальных данных. Ключевой особенностью проекта является реализация гибридной архитектуры безопасности: использование отечественных алгоритмов шифрования сочетается с механизмом делегирования ключей ядру операционной системы (Linux Kernel Keyring).

Основные результаты работы:

- Реализовано импортозамещение: Внедрены алгоритмы “Кузнечик” и “Стрибог”, обеспечивающие стойкость к криптоанализу и соответствие требованиям регуляторов РФ.
- Обеспечена защита от атак на память (Data-in-Use): Решена проблема “остаточной информации” в Python. Ключи шифрования хранятся в изолированном пространстве ядра и принудительно удаляются из памяти процесса, что делает неэффективными атаки типа Cold Boot и Memory Dump.
- Внедрен контроль целостности: реализован механизм Encrypt-then-MAC с использованием имитовставки (HMAC), что защищает базу данных от скрытой модификации и подмены.

Дальнейшие перспективы проекта:

- Кроссплатформенность: Адаптация механизма защиты памяти для ОС Windows с использованием DPAPI (Data Protection API) для обеспечения аналогичного уровня безопасности.
- Поддержка аппаратных токенов: интеграция с российскими токенами (Рутокен, JaCarta) для двухфакторной аутентификации при входе.

- Сетевая синхронизация: Разработка защищенного сервера для синхронизации паролей между устройствами пользователя по зашифрованному каналу (TLS + ГОСТ).

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1) Указ Президента РФ от 01.05.2022 г. № 250 «О дополнительных мерах по обеспечению информационной безопасности Российской Федерации» // Собрание законодательства РФ. - 2022. - № 18. - Ст. 3054.
- 2) ГОСТ Р 34.12–2015. Информационная технология. Криптографическая защита информации. Блочные шифры. - Введ. 2016-01-01. - М.: Стандартинформ, 2015. - 24 с.
- 3) ГОСТ Р 34.11-2012. Информационная технология. Криптографическая защита информации. Функция хэширования. - Введ. 2013-01-01. - М.: Стандартинформ, 2012. - 26 с.
- 4) Р 50.1.113-2016. Информационная технология. Криптографическая защита информации. Криптографические алгоритмы, сопутствующие применению алгоритмов блочного шифрования. - М.: Стандартинформ, 2016.
- 5) Бабаш, А. В. Криптографические методы защиты информации: учебник для вузов / А. В. Бабаш, Г. П. Шанкин. - М.: РИОР: ИНФРА-М, 2023. - 302 с.
- 6) Kerrisk, M. The Linux Programming Interface: A Linux and UNIX System Programmer's Handbook / M. Kerrisk. - San Francisco: No Starch Press, 2010. - 1552 p.
- 7) Halderman, J. A. Lest We Remember: Cold Boot Attacks on Encryption Keys / J. A. Halderman, S. D. Schoen, N. Heninger [et al.] // Proceedings of the 17th USENIX Security Symposium. - 2008. - P. 45–60.
- 8) MITRE ATT&CK Matrix for Enterprise [Электронный ресурс] // MITRE Corporation. - URL: <https://attack.mitre.org/> (дата обращения: 10.01.2026).
- 9) Linux Kernel Documentation. Keys: Keyrings and Key Management [Электронный ресурс] // Kernel.org. - URL: <https://www.kernel.org/doc/html/latest/security/keys/core.html> (дата обращения: 09.01.2026).
- 10) Python 3.14.2 Documentation. ctypes - A foreign function library for Python [Электронный ресурс] // Python Software Foundation. - URL: <https://docs.python.org/3/library/ctypes.html> (дата обращения: 09.01.2026).
- 11) Drobotun, E. GOST crypto 34.10-2012, 34.11-2012 algorithms (Python implementation) [Электронный ресурс] // GitHub Repository. - URL: <https://github.com/drobotun/gostcrypto> (дата обращения: 08.12.2025).
- 12) KeePassXC Password Manager. Official Website [Электронный ресурс]. - URL: <https://keepassxc.org/> (дата обращения: 08.12.2025).

- 13) Kaspersky Password Manager. Технические требования и описание [Электронный ресурс]. - URL: <https://www.kaspersky.ru/password-manager> (дата обращения: 08.12.2025).
- 14) Bitwarden Security Whitepaper [Электронный ресурс]. - URL: <https://bitwarden.com/help/security-whitepaper/> (дата обращения: 08.12.2025).
- 15) Linux Man Pages. keyutils(7) - Linux Key Management Utilities [Электронный ресурс]. - URL: <https://man7.org/linux/man-pages/man7/keyutils.7.html> (дата обращения: 09.01.2026).
- 16) Nikkel, B. Practical Linux Forensics: A Guide for Digital Investigators. - San Francisco: No Starch Press, 2021. - 368 p.
- 17) Linux Man Pages. mlockall(2) - lock process address space [Электронный ресурс]. - URL: <https://man7.org/linux/man-pages/man2/mlockall.2.html> (дата обращения: 11.01.2026)

ПРИЛОЖЕНИЕ

Программный код продукта

Файл main.py

```
from gui import password_manager_app
from utils import lock_memory

if __name__ == "__main__":
    lock_memory()
    app = password_manager_app()
```

Файл utils.py

```
import ctypes
import gc

#ram cleaner
def data_clean(data):
    if isinstance(data, bytearray):
        # (c_char * len)
        buffer = (ctypes.c_char * len(data)).from_buffer(data)
        # Зануляем буфер
        ctypes.memset(buffer, 0, len(data))
    del data
    gc.collect()

#защита от выгрузки в swap
def lock_memory():
    try:
        MCL_CURRENT = 1
        MCL_FUTURE = 2
        libc = ctypes.CDLL("libc.so.6")
        result = libc.mlockall(MCL_CURRENT | MCL_FUTURE)
        if result != 0:
            raise OSError("mlockall failed")
    except Exception as e:
        print(f"{e}")
```

Файл linux_keyring.py

```
import ctypes
import os

try:
    _lib = ctypes.CDLL("libkeyutils.so.1")
except OSError:
    try:
        _lib = ctypes.CDLL("libkeyutils.so")
```

```

    except OSError:
        raise OSError("libkeyutils not found")

key_serial_t = ctypes.c_int32

KEY_SPEC_THREAD_KEYRING = -1
KEY_SPEC_PROCESS_KEYRING = -2
KEY_SPEC_SESSION_KEYRING = -3
KEY_SPEC_USER_KEYRING = -4
KEY_SPEC_USER_SESSION_KEYRING = -5


_lib.add_key.argtypes = [ctypes.c_char_p, ctypes.c_char_p, ctypes.c_void_p,
ctypes.c_size_t, key_serial_t]
_lib.add_key.restype = key_serial_t

_lib.request_key.argtypes = [ctypes.c_char_p, ctypes.c_char_p, ctypes.c_char_p,
key_serial_t]
_lib.request_key.restype = key_serial_t

_lib.keyctl_read.argtypes = [key_serial_t, ctypes.c_char_p, ctypes.c_size_t]
_lib.keyctl_read.restype = ctypes.c_long

_lib.keyctl_revoke.argtypes = [key_serial_t]
_lib.keyctl_revoke.restype = ctypes.c_long


class KeyringManager:
    @staticmethod
    def add(description: str, payload: bytes, keyring=KEY_SPEC_SESSION_KEYRING)
    -> int:
        desc_b = description.encode('utf-8')
        if isinstance(payload, bytearray):
            c_payload = (ctypes.c_char * len(payload)).from_buffer(payload)
        else:
            c_payload = payload
        key_id = _lib.add_key(b"user", desc_b, c_payload, len(payload), keyring)
        if key_id == -1:
            err = ctypes.get_errno()
            raise OSError(err, os.strerror(err))
        return key_id

    @staticmethod
    def read(key_id: int) -> bytearray:
        size = _lib.keyctl_read(key_id, None, 0)
        if size == -1:
            return bytearray()

        buffer = ctypes.create_string_buffer(size)
        ret = _lib.keyctl_read(key_id, buffer, size)
        if ret == -1:
            return bytearray()
        try:
            return bytearray(buffer.raw[:ret])
        finally:
            ctypes.memset(buffer, 0, size)

    @staticmethod

```

```
def revoke(key_id: int):
    _lib.keyctl_revoke(key_id)
```

Файл gui.py

```
import os
import secrets
import string
import tkinter as tk
from tkinter import ttk
from tkinter import messagebox
import pyperclip
from backend import gost_vault
from utils import data_clean
import ctypes
import gc
```

```
DB_FILE = "passwords"
GEN_FILE = "generator"
```

```
#secure tk entry
```

```
class SecureEntry(tk.Entry):
```

```
    def __init__(self, parent, mask_input=True, max_len=64, *args, **kwargs):
        super().__init__(parent, *args, **kwargs)
```

```
        self.MAX_LEN = max_len
        self.mask_input = mask_input
```

```
        self._buffer = bytearray(self.MAX_LEN)
        self._data_len = 0
```

```
        self._char_sizes = []
```

```
        self.bind('<Key>', self.on_key_press)
```

```
        self.bind('<Control-c>', lambda e: "break")
        self.bind('<Control-x>', lambda e: "break")
        self.bind('<<Copy>>', lambda e: "break")
        self.bind('<<Cut>>', lambda e: "break")
```

```
        self.bind('<<Paste>>', self.handle_paste)
        self.bind('<Control-v>', self.handle_paste)
```

```
        self.bind('<Button-3>', lambda e: "break")
```

```
def get_cursor_position(self):
    return self.index(tk.INSERT)
```

```
def _wipe_tail(self):
```

```
    if self._data_len < self.MAX_LEN:
        offset = self._data_len
        length = self.MAX_LEN - self._data_len
        # получаем указатель на начало хвоста
        ptr = (ctypes.c_char * length).from_buffer(self._buffer, offset)
```

```

        ctypes.memset(ptr, 0, length)

    def _shift_buffer_right(self, byte_idx, shift_amount):
        if self._data_len + shift_amount > self.MAX_LEN:
            return False # переполнение

        count = self._data_len - byte_idx
        if count > 0:
            src_addr = (ctypes.c_char * count).from_buffer(self._buffer,
byte_idx)
            dst_addr = (ctypes.c_char * count).from_buffer(self._buffer,
byte_idx + shift_amount)
            ctypes.memmove(dst_addr, src_addr, count)

        return True

    def _shift_buffer_left(self, byte_idx, shift_amount):
        count = self._data_len - (byte_idx + shift_amount)
        if count > 0:
            src_addr = (ctypes.c_char * count).from_buffer(self._buffer,
byte_idx + shift_amount)
            dst_addr = (ctypes.c_char * count).from_buffer(self._buffer,
byte_idx)
            ctypes.memmove(dst_addr, src_addr, count)

        self._data_len -= shift_amount
        self._wipe_tail()

    def _get_byte_offset(self, char_index):
        return sum(self._char_sizes[:char_index])

    def _insert_text_at_cursor(self, text):
        if not text: return

        total_new_bytes = sum(len(c.encode('utf-8')) for c in text)
        if self._data_len + total_new_bytes > self.MAX_LEN:
            return

        char_pos = self.get_cursor_position()

        if self.selection_present():
            self._delete_selection()
            char_pos = self.get_cursor_position()

        byte_pos = self._get_byte_offset(char_pos)

        if not self._shift_buffer_right(byte_pos, total_new_bytes):
            return

        current_byte_pos = byte_pos
        for char in text:
            char_bytes = char.encode('utf-8')
            b_len = len(char_bytes)

            for i, b in enumerate(char_bytes):
                self._buffer[current_byte_pos + i] = b

```

```

        self._char_sizes.insert(char_pos, b_len)
        char_pos += 1
        current_byte_pos += b_len

    self._data_len += total_new_bytes

    display_text = "" * len(text) if self.mask_input else text
    self.insert(tk.INSERT, display_text)

    del text
    gc.collect()

def _delete_selection(self):
    try:
        start_char = self.index(tk.SEL_FIRST)
        end_char = self.index(tk.SEL_LAST)

        start_byte = self._get_byte_offset(start_char)

        bytes_to_del = sum(self._char_sizes[start_char:end_char])

        self._shift_buffer_left(start_byte, bytes_to_del)

        del self._char_sizes[start_char:end_char]

        self.delete(start_char, end_char)
    except tk.TclError:
        pass

def _backspace(self):
    if self.selection_present():
        self._delete_selection()
        return

    pos = self.get_cursor_position()
    if pos > 0:
        # удаляем 1 символ слева от курсора
        char_len = self._char_sizes[pos-1]
        byte_start = self._get_byte_offset(pos-1)

        self._shift_buffer_left(byte_start, char_len)
        del self._char_sizes[pos-1]
        self.delete(pos-1)

def _delete_key(self):
    if self.selection_present():
        self._delete_selection()
        return

    pos = self.get_cursor_position()
    if pos < len(self._char_sizes):
        # удаляем 1 символ справа от курсора
        char_len = self._char_sizes[pos]
        byte_start = self._get_byte_offset(pos)

        self._shift_buffer_left(byte_start, char_len)
        del self._char_sizes[pos]
        self.delete(pos)

```

```

def handle_paste(self, event):
    try:
        text = self.clipboard_get()
        if text:
            self._insert_text_at_cursor(text)
    except tk.TclError:
        pass
    return "break"

def on_key_press(self, event):
    key = event.keysym

    if key in ('Left', 'Right', 'Up', 'Down', 'Home', 'End',
               'Shift_L', 'Shift_R', 'Control_L', 'Control_R',
               'Alt_L', 'Alt_R', 'Caps_Lock', 'Num_Lock',
               'Return', 'Tab', 'Escape'):
        return None

    if key == 'BackSpace':
        self._backspace()
        return "break"

    if key == 'Delete':
        self._delete_key()
        return "break"

    # ВВОД СИМВОЛОВ
    if len(event.char) > 0 and ord(event.char) >= 32:
        self._insert_text_at_cursor(event.char)
        del event
        gc.collect()
        return "break"

    return "break"

def get_bytes(self):
    return self._buffer[:self._data_len]

def clear(self):
    # жесткая очистка памяти
    ptr = (ctypes.c_char * self.MAX_LEN).from_buffer(self._buffer)
    ctypes.memset(ptr, 0, self.MAX_LEN)

    self._data_len = 0
    self._char_sizes = []
    self.delete(0, tk.END)

def set_text(self, text):
    self.clear()
    self._insert_text_at_cursor(text)

# GUI
class password_manager_app:
    def __init__(self):
        self.root = tk.Tk()
        self.root.withdraw()

```

```

self.pwd_buffer = None

auth_window = tk.Toplevel(self.root)
auth_window.title("авторизация")
auth_window.config(padx=50, pady=20)
tk.Label(auth_window, text="введите пароль:").pack(pady=10)

# наш виджет
secure_entry = SecureEntry(auth_window, width=20)
secure_entry.pack(pady=5)
secure_entry.focus_set()

# получаем мастер пароль
def on_submit(event=None):
    self.pwd_buffer = secure_entry.get_bytes()[:]
    secure_entry.clear()
    auth_window.destroy()

btn = tk.Button(auth_window, text="OK", command=on_submit)
btn.pack(pady=10)
auth_window.bind('<Return>', on_submit)

self.root.wait_window(auth_window)

if not self.pwd_buffer:
    self.root.destroy()
    return

try:
    salt = None
    if os.path.exists(DB_FILE):
        with open(DB_FILE, "rb") as f:
            header = f.read(32)
            if len(header) == 32:
                salt = header

    self.vault = gost_vault(self.pwd_buffer, salt=salt)

    self.curr_data = self.load_db()
except Exception as e:
    messagebox.showerror("Ошибка", e)
finally:
    data_clean(self.pwd_buffer)

if self.curr_data is None:
    messagebox.showerror("---", "неверный пароль")
    self.root.destroy()
    return

self.load_config()

self.setup_ui()
self.root.deiconify()
self.root.protocol("WM_DELETE_WINDOW", self.on_close)
self.root.mainloop()

def on_close(self):
    if self.vault:

```

```

        self.vault.cleanup()
self.root.destroy()

def load_config(self):
    # инициализация настроек генератора
    try:
        if os.path.exists(GEN_FILE):
            with open(GEN_FILE, "r") as f:
                settings = f.read()
                settings = settings.split()
                self.generator_len = int(settings[0])
                self.generator_chars = settings[1]
            else:
                self.generator_len = 17
                self.generator_chars = "all"
    except:
        print(f"файл настроек битый")
        self.generator_len = 17
        self.generator_chars = "all"

def load_db(self):
    if not os.path.exists(DB_FILE):
        return {}

    try:
        with open(DB_FILE, "rb") as f:
            content = f.read()

            if len(content) < 32:
                return {}
            content = content[32:]

            #загрузка и шифрование паролей
            ram_data = {}
            for site, creds in self.vault.decrypt_data(content).items():
                pwd_enc = self.vault.encrypt_bytes(f"{site}|{creds['password']}".encode('utf-8'), init_data=site)
                ram_data[site] = {
                    "email": creds['email'],
                    "password": pwd_enc
                }

            return ram_data
    except Exception as e:
        print(f"error: {e}")
        return None

def save_db(self):
    try:
        #выгрузка и расшифровка паролей
        data = {}
        for site, creds in self.curr_data.items():
            data[site] = {
                "email": creds['email'],
                "password": self.vault.decrypt_bytes(creds['password'],
init_data=site).split("|", 1)[1]
            }

        enc_bytes = self.vault.encrypt_data(data)

```

```

        with open(DB_FILE, "wb") as f:
            f.write(self.vault.salt + enc_bytes)
        return True
    except Exception as e:
        messagebox.showerror("error", str(e))
        return False

# UI handlers

def generator(self):
    if self.generator_chars == "all":
        alph = string.ascii_letters + string.digits + string.punctuation
    elif self.generator_chars == "punc":
        alph = string.ascii_letters + string.punctuation
    elif self.generator_chars == "digs":
        alph = string.ascii_letters + string.digits
    elif self.generator_chars == "let":
        alph = string.ascii_letters

    pwd = ''.join(secrets.choice(alph) for _ in range(self.generator_len))

    self.pwd_entry.set_text(pwd)
    pyperclip.copy(pwd)
    messagebox.showinfo("---", "пароль скопирован")

def save_entry(self):
    web = self.web_entry.get()
    email = self.email_entry.get()
    pwd = self.pwd_entry.get_bytes()

    if len(web) == 0 or len(pwd) == 0:
        messagebox.showwarning("---", "впишите сайт и пароль.")
        return

    pwd_enc = self.vault.encrypt_bytes(web.encode("utf-8") + b"| " + pwd,
init_data=web)
    data_clean(pwd)
    self.pwd_entry.clear()

    self.curr_data[web] = {
        "email": email,
        "password": pwd_enc
    }

    if self.save_db():
        self.web_entry.delete(0, tk.END)
        self.pwd_entry.delete(0, tk.END)
        messagebox.showinfo("---", f"пароль сохранен")

def find_pwd(self):
    web = self.web_entry.get()
    if web in self.curr_data:
        email = self.curr_data[web]["email"]
        pwd_enc = self.curr_data[web]["password"]

        try:
            pwd = self.vault.decrypt_bytes(pwd_enc,
init_data=web).split("|", 1)[1]

```

```

        pyperclip.copy(pwd)
        messagebox.showinfo(web, f"email: {email}\nпароль: {pwd}
\n\nпароль скопирован в буфер обмена")
    finally:
        #чистим переменные
        if "pwd" in locals():
            data_clean(pwd)
            gc.collect()
    else:
        messagebox.showerror("---", f"данных не найдено")

def save_config(self):
    if self.generator_chars_sel.get() == "буквы + цифры":
        self.gen_chars = "digs"
    elif self.generator_chars_sel.get() == "буквы + пунктуация":
        self.gen_chars = "punc"
    elif self.generator_chars_sel.get() == "буквы + цифры + пунктуация":
        self.gen_chars = "all"

    with open(GEN_FILE, "w") as f:
        f.write(f"{self.generator_len_sel.get()} {self.gen_chars}")

def password_generator_config(self):
    conf_window = tk.Toplevel(self.root)
    conf_window.config(padx=50, pady=50)
    conf_window.title("конфиг генератора")
    self.generator_chars_sel = tk.StringVar()
    self.generator_len_sel = 17

    tk.Label(conf_window, text="выбор символов:").grid(column=0, row=0)
    tk.Label(conf_window, text="выбор длины пароля:").grid(column=0, row=1)

    choices = ["буквы + цифры", "буквы + пунктуация", "буквы + цифры +
пунктуация"]
    ttk.Combobox(conf_window, width=36,
textvariable=self.generator_chars_sel, values=choices).grid(column=1, row=0,
padx=5)

    self.generator_len_sel = tk.Entry(conf_window, width=36)
    self.generator_len_sel.grid(column=1, row=1)
    self.generator_len_sel.insert(0, "17")
    self.generator_len_sel.focus()

    tk.Button(conf_window, text="сохранить конфиг",
command=self.save_config).grid(column=0, columnspan=2, row=2, pady=10, padx=5,
sticky="EW")

def db_delete(self):
    ans = messagebox.askyesno("", "Это действие не обратимо, вы уверены?")
    if not ans:
        return
    os.remove(DB_FILE)
    data_clean(self.curr_data)
    self.curr_data = {}
def change_db_pwd(self):
    print(1)

```

```

def setup_ui(self):
    self.root.title("менеджер паролей")
    self.root.config(padx=50, pady=50)

    tk.Label(text="веб-сайт:").grid(column=0, row=1)
    tk.Label(text="email/логин:").grid(column=0, row=2)
    tk.Label(text="пароль:").grid(column=0, row=3)

    self.web_entry = tk.Entry(width=35)
    self.web_entry.grid(column=1, row=1)
    self.web_entry.focus()

    self.email_entry = tk.Entry(width=35)
    self.email_entry.grid(column=1, row=2, columnspan=2, sticky="EW")
    self.email_entry.insert(0, "aboba@example.com")

    self.pwd_entry = SecureEntry(self.root, mask_input=False, width=21)
    self.pwd_entry.grid(column=1, row=3, sticky="EW")

    tk.Button(text="найти", command=self.find_pwd).grid(column=2, row=1,
    sticky="EW")
    tk.Button(text="сгенерировать", command=self.generator).grid(column=2,
    row=3, sticky="EW")
    tk.Button(text="добавить", width=36,
    command=self.save_entry).grid(column=1, row=4, columnspan=2, sticky="EW")

    tk.Button(text="настроить генератор пароля", width=18,
    command=self.password_generator_config).grid(column=1, row=5, padx=5, pady=10,
    columnspan=1, sticky="EW")
    tk.Button(text="удалить базу", command=self.db_delete).grid(column=2,
    row=5, padx=5, pady=10, sticky="EW")

```

Файл backend.py

```

import json
import os
import hmac
from linux_keyring import KeyringManager, KEY_SPEC_PROCESS_KEYRING

from gostcrypto import gostcipher, gosthmac, gostpbkdf, gosthash

from utils import data_clean

# backend logic
class gost_vault:
    def __init__(self, paster_pwd, salt=None):
        if salt is None:
            self.salt = os.urandom(32)
        else:
            self.salt = salt

```

```

        # PBKDF
        pbkdf_obj = gostpbkdf.new(password=paster_pwd, salt=self.salt,
counter=10**3)
        key_mats = bytearray(pbkdf_obj.derive(64))

        self.key_enc = key_mats[:32] #32 bytes
        self.key_mac = key_mats[32:]
        #передача ключей на хранение ядру
        self.key_enc_id = KeyringManager.add("gost_enc", self.key_enc,
KEY_SPEC_PROCESS_KEYRING)
        self.key_mac_id = KeyringManager.add("gost_mac", self.key_mac,
KEY_SPEC_PROCESS_KEYRING)

        data_clean(self.key_enc)
        data_clean(self.key_mac)

        # kuznechik block - 16 byte(128 bit), mac size 32 bytes(256 bit)
        self.kuzn_size = 16
        self.mac_size = 32

        #получение ключей из ядра
        def _get_key_enc(self):
            return KeyringManager.read(self.key_enc_id)
        def _get_key_mac(self):
            return KeyringManager.read(self.key_mac_id)
        #PKCS7
        def _pad(self, data):
            pad_len = self.kuzn_size - (len(data) % self.kuzn_size)
            return data + bytes([pad_len] * pad_len)

        def _unpad(self, data):
            if not data:
                return b""
            pad_len = data[-1]
            return data[:-pad_len]

        def encrypt_data(self, data):
            json_bytes = json.dumps(data).encode('utf-8')
            pad_data = self._pad(json_bytes)
            iv = os.urandom(self.kuzn_size)

            self.key_enc = self._get_key_enc()
            try:
                kuznechik = gostcipher.new('kuznechik', self.key_enc,
gostcipher.MODE_CBC, iv=iv)
                enc_data = kuznechik.encrypt(pad_data)
            finally:
                data_clean(self.key_enc)

            # создание имитовставки (HMAC)
            data = iv + enc_data
            self.key_mac = self._get_key_mac()
            try:
                if self.mac_size == 32:
                    hmac_obj = gosthmac.new('HMAC_GOSTR3411_2012_256', self.key_mac,
data=data)
                elif self.mac_size == 64:
                    hmac_obj = gosthmac.new('HMAC_GOSTR3411_2012_512', self.key_mac,
data=data)

```

```

        mac = hmac_obj.digest()
    finally:
        data_clean(self.key_mac)

    return iv + mac + enc_data

#генерация iv для шифрования паролей
def _generate_iv_simple(self, init_data):
    if isinstance(init_data, str):
        data = init_data.encode('utf-8')
    else:
        data = init_data

    hasher = gosthash.new('streebog256', data=data)
    digest = hasher.digest()
    return digest[:self.kuzn_size]

def decrypt_data(self, enc_bytes):
    data_len = self.kuzn_size + 16 + self.mac_size # data 1 block(16 bytes)
    + iv(16 bytes) + mac(32 bytes)
    if len(enc_bytes) < data_len:
        raise ValueError("файл поврежден")
    # парсинг данных из файла
    iv = enc_bytes[:self.kuzn_size]
    mac = enc_bytes[self.kuzn_size:(self.kuzn_size+self.mac_size)]
    enc_data = enc_bytes[(self.kuzn_size+self.mac_size):]
    self.key_mac = self._get_key_mac()
    self.key_enc = self._get_key_enc()
    #data check
    data = iv + enc_data
    if self.mac_size == 32:
        hmac_obj = gosthmac.new('HMAC_GOSTR3411_2012_256', self.key_mac,
data=data)
    elif self.mac_size == 64:
        hmac_obj = gosthmac.new('HMAC_GOSTR3411_2012_512', self.key_mac,
data=data)
    calc_mac = hmac_obj.digest()

    # Encrypt-then-MAC для защиты от Chosen-Ciphertext Attacks (CCA)
    if not(hmac.compare_digest(calc_mac, mac)):
        raise ValueError("Неправильный пароль или данные повреждены")

    kuznechik = gostcipher.new('kuznechik', self.key_enc,
gostcipher.MODE_CBC, iv=iv)
    data_clean(self.key_enc)
    data_clean(self.key_mac)
    try:
        return
    json.loads(self._unpad(kuznechik.decrypt(enc_data)).decode('utf-8'))
    except:
        return None
#шифрование отдельных данных
def encrypt_bytes(self, data, init_data=None):
    self.key_enc = self._get_key_enc()
    try:
        if init_data is None:
            kuznechik = gostcipher.new('kuznechik', self.key_enc,
gostcipher.MODE_ECB)
            return kuznechik.encrypt(self._pad(data))
        else:

```

```

        iv_d = self._generate_iv_simple(init_data=init_data)

        kuznechik = gostcipher.new('kuznechik', self.key_enc,
gostcipher.MODE_CBC, iv=iv_d)
        return kuznechik.encrypt(self._pad(data))
    finally:
        data_clean(self.key_enc)

#расшифровываем отдельные данные
    def decrypt_bytes(self, enc_bytes, init_data=None):
        self.key_enc = self._get_key_enc()
        try:
            if init_data is None:
                kuznechik = gostcipher.new('kuznechik', self.key_enc,
gostcipher.MODE_ECB)
                return self._unpad(kuznechik.decrypt(enc_bytes)).decode("utf-8")
            else:
                iv_d = self._generate_iv_simple(init_data=init_data)

                kuznechik = gostcipher.new('kuznechik', self.key_enc,
gostcipher.MODE_CBC, iv=iv_d)
                return self._unpad(kuznechik.decrypt(enc_bytes)).decode("utf-8")
        finally:
            data_clean(self.key_enc)

    def cleanup(self):
        try:
            KeyringManager.revoke(self.key_enc_id)
            KeyringManager.revoke(self.key_mac_id)
        except Exception as e:
            print(f"error: {e}")
        pass

```